

## 🔗 Advanced Functions — Closure থেকে Advanced Function Patterns

Chapter 6-এ আমরা Function-এর basic concept এবং Chapter 25-এ Functional JavaScript শিখেছি। এখন সেই foundation-এর ওপর দাঁড়িয়ে JavaScript-এর **সবচেয়ে গুরুত্বপূর্ণ advanced function concepts**—Scope Chain, Closures, IIFE, Currying, Function Composition, Pipelines, Memoization এবং Throttling/Debouncing—নিখুঁতভাবে শিখব।

- |                           |                              |                              |
|---------------------------|------------------------------|------------------------------|
| ★ 1. Scope Recap          | ★ 2. Lexical Scope           | ★ 3. Scope Chain             |
| ★ 4. Closure Mechanics    | ★ 5. Private Variables       | ★ 6. Data Encapsulation      |
| ★ 7. Factory Functions    | ★ 8. IIFE Patterns           | ★ 9. Private State (IIFE)    |
| ★ 10. Returning Functions | ★ 11. Currying (N-ary)       | ★ 12. Partial Application    |
| ★ 13. compose() Pipeline  | ★ 14. pipe() Flow            | ★ 15. Memoization Cache      |
| ★ 16. Function Decorators | ★ 17. Execution Timing       | ★ 18. Debounce Logic         |
| ★ 19. Throttle Interval   | ★ 20. Loop Binding (let/var) | ★ 21. Memory Management      |
| ★ 22. Permission Factory  | ★ 23. ID Generator           | ★ 24. Mini Project (Counter) |

### Part 01 — Scope, Lexical Environment & Scope Chain (26.1 – 26.3)

#### 26.1 Scope কী?

**Scope** হলো কোনো variable কোথা থেকে access করা যাবে তার সীমা। JavaScript-এ প্রধানত:

🕒 ARCHITECTURE / CONCEPT FLOW

Global Scope  
Function Scope  
Block Scope  
Module Scope

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
let name = "Ripon";

function greet() {
  console.log(name);
}

greet();
```

▶ CONSOLE OUTPUT

OUTPUT

Ripon

কারণ `greet()` function তার বাইরের scope-এর `name` access করতে পারছে।

## 26.2 Lexical Scope

JavaScript-এ function **যেখানে লেখা হয়েছে**, সেই জায়গার scope-এর ওপর তার access নির্ভর করে। এটাকে বলা হয়: > **Lexical Scope** Example:

JAVASCRIPT SOURCE JAVASCRIPT

```
const name = "Ripon";

function outer() {

  function inner() {
    console.log(name);
  }

  inner();
}

outer();
```

▶ CONSOLE OUTPUT OUTPUT

```
Ripon
```

`inner()` তার parent scope এবং তার বাইরের scope থেকে variable খুঁজে নেয়।

## 26.3 Scope Chain

যখন JavaScript কোনো variable খুঁজে, তখন সাধারণত:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Current Scope
  ↓
Parent Scope
  ↓
Grandparent Scope
  ↓
Global Scope
```

এভাবে খুঁজতে থাকে। Example:

JAVASCRIPT SOURCE JAVASCRIPT

```
const a = 10;

function outer() {
  const b = 20;

  function inner() {
    const c = 30;

    console.log(a);
    console.log(b);
    console.log(c);
  }

  inner();
}

outer();
```

▶ CONSOLE OUTPUT OUTPUT

```
10
20
30
```

## Part 02 — Closures, Private State, Factories & Loop Bindings (26.4 – 26.19)

## 26.4 Closure কী?

এখন আসি এই chapter-এর সবচেয়ে গুরুত্বপূর্ণ concept-এ। > **Closure হলো এমন একটি function**, যা তার lexical scope-এর variable-গুলোকে মনে রাখতে পারে, এমনকি **outer function execution শেষ হয়ে যাওয়ার পরেও**। সহজভাবে:

🕒 ARCHITECTURE / CONCEPT FLOW

```
Outer Function
  ↓
Inner Function
  ↓
Inner function remembers outer variables
```

## 26.5 প্রথম Closure Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
function outer() {  
  let message = "Hello";  
  function inner() {  
    console.log(message);  
  }  
  return inner;  
}  
  
const greet = outer();  
  
greet();
```

► CONSOLE OUTPUT

OUTPUT

Hello

এখানে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
outer();
```

শেষ হয়ে গেছে। তারপরও:

JAVASCRIPT SOURCE

JAVASCRIPT

```
greet();
```

`message` access করতে পারছে। **এটাই Closure!**

## 26.6 কীভাবে কাজ করছে?

এই code:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function outer() {  
  
  let message = "Hello";  
  
  function inner() {  
    console.log(message);  
  }  
  
  return inner;  
}
```

যখন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const greet = outer();
```

হয়:

🕒 ARCHITECTURE / CONCEPT FLOW

```
outer()  
↓  
message = "Hello"  
↓  
inner function তৈরি  
↓  
inner return  
↓  
greet → inner function
```

তারপর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
greet();
```

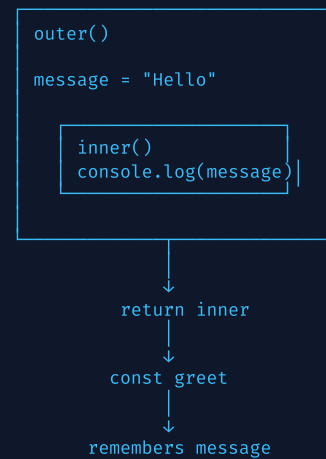
হলে:

🕒 ARCHITECTURE / CONCEPT FLOW

```
greet()  
↓  
inner()  
↓  
message  
↓  
"Hello"
```

## 26.7 Closure Visual Model

🕒 ARCHITECTURE / CONCEPT FLOW



## 26.8 Closure-এর Real-Life Example

ধরো ATM machine। User বাইরে থেকে সরাসরি account balance পরিবর্তন করতে পারবে না।

🕒 ARCHITECTURE / CONCEPT FLOW

```
ATM Account
├── balance
├── deposit()
└── withdraw()
```

এখানে balance private রাখা যায় Closure দিয়ে।

## 26.9 Private Variable with Closure

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createAccount(initialBalance) {
  let balance = initialBalance;

  return {
    deposit(amount) {
      balance += amount;
    },

    withdraw(amount) {
      if (amount ≤ balance) {
        balance -= amount;
      }
    },

    getBalance() {
      return balance;
    }
  };
}
```

এখন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const account = createAccount(1000);
account.deposit(500);
console.log(account.getBalance());
```

▶ CONSOLE OUTPUT

OUTPUT

1500

কিন্তু:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(account.balance);
```

▶ CONSOLE OUTPUT

OUTPUT

undefined

কারণ balance directly accessible নয়।

## 26.10 কেন Closure ব্যবহার করব?

Closure দিয়ে:

- private data রাখা যায়
- state maintain করা যায়
- factory function বানানো যায়
- callback state মনে রাখতে পারে
- event handler state ধরে রাখতে পারে
- memoization করা যায়
- function factory তৈরি করা যায়

## 26.11 Closure দিয়ে Counter

এটি অত্যন্ত গুরুত্বপূর্ণ example।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createCounter() {  
  let count = 0;  
  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const counter = createCounter();  
  
console.log(counter());  
console.log(counter());  
console.log(counter());
```

▶ CONSOLE OUTPUT

OUTPUT

```
1  
2  
3
```

এখানে `count` কোথাও global variable নয়। তবুও function সেটাকে মনে রাখছে।

## 26.12 Multiple Closures

এখন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const counter1 = createCounter();  
const counter2 = createCounter();  
  
console.log(counter1());  
console.log(counter1());  
  
console.log(counter2());  
console.log(counter2());
```

▶ CONSOLE OUTPUT

OUTPUT

```
1  
2  
1  
2
```

কারণ প্রতিটি `createCounter()` আলাদা closure তৈরি করেছে।

🕒 ARCHITECTURE / CONCEPT FLOW

```
counter1 → count = 2  
counter2 → count = 2
```

দুটো আলাদা state।

### 26.13 Closure + Parameters

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createGreeting(name) {  
    return function() {  
        console.log(`Hello ${name}`);  
    };  
}  
  
const greetRipon = createGreeting("Ripon");  
const greetRahim = createGreeting("Rahim");  
  
greetRipon();  
greetRahim();
```

▶ CONSOLE OUTPUT

OUTPUT

```
Hello Ripon  
Hello Rahim
```

প্রতিটি returned function নিজের `name` মনে রেখেছে।

### 26.14 Function Factory

যে function অন্য function তৈরি করে তাকে **Function Factory** বলা যায়। Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createMultiplier(number) {  
    return function(value) {  
        return value * number;  
    };  
}
```

এখন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const double = createMultiplier(2);  
const triple = createMultiplier(3);  
  
console.log(double(10));  
console.log(triple(10));
```

▶ CONSOLE OUTPUT

OUTPUT

```
20  
30
```

### 26.15 Real-Life Function Factory

Discount calculator:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createDiscount(rate) {  
    return function(price) {  
        return price - price * rate;  
    };  
}  
  
const tenPercentOff = createDiscount(0.10);  
const twentyPercentOff = createDiscount(0.20);  
  
console.log(tenPercentOff(1000));  
console.log(twentyPercentOff(1000));
```

▶ CONSOLE OUTPUT

OUTPUT

```
900  
800
```

## 26.16 Closure with setTimeout()

Closure asynchronous JavaScript-এও খুব গুরুত্বপূর্ণ।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function greetLater(name) {  
  
    setTimeout(function() {  
        console.log(`Hello ${name}`);  
    }, 1000);  
}  
  
greetLater("Ripon");
```

এক সেকেন্ড পরে:

▶ CONSOLE OUTPUT

OUTPUT

Hello Ripon

setTimeout() callback name variable-টিকে মনে রেখেছে।

## 26.17 Closure with Event Listener

Browser-এ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function setupButton(name) {  
  
    document  
        .getElementById("btn")  
        .addEventListener("click", function() {  
  
            console.log(`Hello ${name}`);  
  
        });  
}  
  
setupButton("Ripon");
```

Button click করলে:

▶ CONSOLE OUTPUT

OUTPUT

Hello Ripon

এখানেও callback name মনে রেখেছে।

## 26.18 Closure-এর সবচেয়ে Common Example: Loop

JavaScript-এ বহুল পরিচিত একটি ক্লাসিক ট্র্যাপ হলো লুপে var ব্যবহার করে Closure তৈরি করা:

**সতর্কতা:** var কীওয়ার্ড ফাংশন-স্কোপড হওয়ায় সবকটি টাইমআউট কলব্যাক একই ভ্যারিয়েবল রেফারেন্স শেয়ার করে। ফলে লুপ শেষ হয়ে i-এর মান যখন 4 হয়, তখন সব কলব্যাক একই মান 4 প্রিন্ট করে।

একটি classic problem:

✗ LOOP WITH VAR (SHARED REFERENCE)

✗ TRAP / ANTIPATTERN

```
for (var i = 1; i ≤ 3; i++) {  
  
    setTimeout(function() {  
        console.log(i);  
    }, 1000);  
}
```

অনেক ক্ষেত্রে output হবে:

▶ CONSOLE OUTPUT

OUTPUT

4  
4  
4

কারণ var function-scoped এবং callbacks একই variable binding-এর দিকে তাকিয়ে থাকে।

### 26.19 let ব্যবহার করলে

Modern JavaScript-এ (ES6+) ব্লক-স্কোপড `let` ব্যবহার করলে প্রতি লুপ ইটারেশনে একটি করে নতুন লেক্সিক্যাল বাইন্ডিং তৈরি হয়:

✓ LOOP WITH LET (PER-ITERATION LEXICAL BINDING)

✓ BEST PRACTICE

```
for (let i = 1; i ≤ 3; i++) {  
  setTimeout(function() {  
    console.log(i);  
  }, 1000);  
}
```

► CONSOLE OUTPUT

OUTPUT

```
1  
2  
3
```

কারণ `let` loop iteration-এর জন্য আলাদা binding তৈরি করে।

**Key Takeaway:** `let` প্রতিটি ইটারেশনের জন্য ক্লোজারের ভিতরে একটি স্বতন্ত্র মেমরি রেফারেন্স সংরক্ষণ করে, যা প্রত্যাশিত 1, 2, 3 আউটপুট নিশ্চিত করে।

## Part 03 — Immediately Invoked Function Expressions (IIFE) (26.20 – 26.26)

### 26.20 IIFE কী?

**IIFE = Immediately Invoked Function Expression** এমন function expression যা define করার সাথে সাথে execute হয়। Syntax:

JAVASCRIPT SOURCE

JAVASCRIPT

```
(function() {  
  console.log("Hello");  
})();
```

► CONSOLE OUTPUT

OUTPUT

```
Hello
```

### 26.21 IIFE কীভাবে কাজ করে?

সাধারণ function:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function greet() {  
  console.log("Hello");  
}  
  
greet();
```

IIFE:

JAVASCRIPT SOURCE

JAVASCRIPT

```
(function() {  
  console.log("Hello");  
})();
```

এক statement-এর মধ্যেই:

🕒 ARCHITECTURE / CONCEPT FLOW

Function তৈরি

↓

Immediately execute

### 26.22 IIFE with Arrow Function

JAVASCRIPT SOURCE

JAVASCRIPT

```
(() => {  
  console.log("Hello from IIFE");  
})();
```

► CONSOLE OUTPUT

OUTPUT

```
Hello from IIFE
```

### 26.23 IIFE কেন ব্যবহার করা হতো?

আগে JavaScript-এ module system শক্তিশালী হওয়ার আগে global namespace pollution এড়াতে IIFE অনেক ব্যবহার করা হতো। Example:

```
JAVASCRIPT SOURCE JAVASCRIPT
(function() {
  const secret = "12345";
  console.log(secret);
})();
```

বাহিরে:

```
JAVASCRIPT SOURCE JAVASCRIPT
console.log(secret);

▶ CONSOLE OUTPUT OUTPUT
ReferenceError
```

কারণ `secret` IIFE-এর ভিতরের scope-এ আছে। বর্তমানে **ES Modules** অনেক ক্ষেত্রে এই কাজের আরও ভালো solution।

### 26.24 IIFE with Parameters

```
JAVASCRIPT SOURCE JAVASCRIPT
(function(name) {
  console.log(`Hello ${name}`);
})("Ripon");

▶ CONSOLE OUTPUT OUTPUT
Hello Ripon
```

### 26.25 IIFE দিয়ে Private State

```
JAVASCRIPT SOURCE JAVASCRIPT
const counter = (function() {
  let count = 0;

  return {
    increment() {
      count++;
    },
    getCount() {
      return count;
    }
  };
})();
```

ব্যবহার:

```
JAVASCRIPT SOURCE JAVASCRIPT
counter.increment();
counter.increment();

console.log(counter.getCount());

▶ CONSOLE OUTPUT OUTPUT
2
```

এখানে `count` বাহিরে থেকে directly accessible নয়।

## 26.26 Closure vs IIFE

| Closure                             | IIFE                         |
|-------------------------------------|------------------------------|
| Scope মনে রাখে                      | Immediately execute হয়      |
| সাধারণত returned function দেখা যায় | Function immediately run করে |
| Private state তৈরি করতে পারে        | Private scope তৈরি করতে পারে |
| Long-lived হতে পারে                 | একবার execute হওয়া common   |
| createCounter() example             | (function(){})()             |

একটি গুরুত্বপূর্ণ বিষয়: > **IIFE এবং Closure একই জিনিস নয়।** IIFE হলো execution pattern। Closure হলো lexical environment ধরে রাখার behavior। একটি IIFE closure তৈরি করতেও পারে।

## Part 04 — Currying, Partial Application & Function Transformation (26.27 – 26.33)

### 26.27 Function Returning Function

JAVASCRIPT SOURCE

JAVASCRIPT

```
function outer() {  
  return function inner() {  
    return "Hello";  
  };  
}  
  
const result = outer();  
console.log(result());
```

► CONSOLE OUTPUT

OUTPUT

Hello

এটি closure, factory এবং currying-এর foundation।

### 26.28 Currying

Chapter 25-এ basic দেখেছিলাম। এবার গভীরভাবে বুঝি। Normal function:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a, b, c) {  
  return a + b + c;  
}  
  
console.log(add(10, 20, 30));
```

Curried:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function add(a) {  
  return function(b) {  
    return function(c) {  
      return a + b + c;  
    };  
  };  
}  
  
console.log(add(10)(20)(30));
```

► CONSOLE OUTPUT

OUTPUT

60

### 26.29 Arrow Currying

JAVASCRIPT SOURCE

JAVASCRIPT

```
const add = a => b => c => a + b + c;  
console.log(add(10)(20)(30));
```

▶ CONSOLE OUTPUT

OUTPUT

60

### 26.30 Currying-এর সুবিধা

ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const add = a => b => c => a + b + c;
```

আমরা partial function তৈরি করতে পারি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const add10 = add(10);
```

তারপর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(add10(20)(30));
```

▶ CONSOLE OUTPUT

OUTPUT

60

### 26.31 Practical Currying — Tax

JAVASCRIPT SOURCE

JAVASCRIPT

```
const calculateTax = rate => price =>  
  price * rate;
```

এখন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const vat = calculateTax(0.15);  
console.log(vat(1000));  
console.log(vat(2000));
```

▶ CONSOLE OUTPUT

OUTPUT

150  
300

### 26.32 Partial Application

Partial application হলো কিছু argument আগে দিয়ে একটি নতুন function তৈরি করা।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function multiply(a, b, c) {  
  return a * b * c;  
}  
  
function multiplyBy(a) {  
  return function(b, c) {  
    return multiply(a, b, c);  
  };  
}  
  
const multiplyBy2 = multiplyBy(2);  
  
console.log(multiplyBy2(3, 4));
```

► CONSOLE OUTPUT

OUTPUT

24

### 26.33 Currying vs Partial Application

| Currying                | Partial Application                      |
|-------------------------|------------------------------------------|
| একবারে একটি argument    | কিছু argument আগে দেওয়া যায়            |
| $f(a)(b)(c)$            | $f(a) \rightarrow \text{remaining args}$ |
| Function transformation | Pre-configured function                  |

## Part 05 — Function Composition, Pipelines, Memoization & Rate Limiting (26.34 – 26.53)

### 26.34 Function Composition

একটি function-এর output আরেকটি function-এর input হলে composition করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const double = x => x * 2;  
const square = x => x * x;  
  
console.log(square(double(5)));
```

🕒 ARCHITECTURE / CONCEPT FLOW

```
5  
↓  
double  
↓  
10  
↓  
square  
↓  
100
```

### 26.35 compose() Function

JAVASCRIPT SOURCE

JAVASCRIPT

```
const compose = (f, g) => value =>
  f(g(value));
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const double = x => x * 2;
const square = x => x * x;

const process = compose(square, double);

console.log(process(5));
```

▶ CONSOLE OUTPUT

OUTPUT

100

এখানে:

🕒 ARCHITECTURE / CONCEPT FLOW

```
process(5)
  ↓
double(5)
  ↓
10
  ↓
square(10)
  ↓
100
```

### 26.36 Multiple Function Compose

একাধিক function handle করার জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const compose = (...functions) => value =>
  functions.reduceRight(
    (result, fn) => fn(result),
    value
  );
```

এখন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const double = x => x * 2;
const add10 = x => x + 10;
const square = x => x * x;

const process = compose(
  square,
  add10,
  double
);

console.log(process(5));
```

▶ CONSOLE OUTPUT

OUTPUT

400

Execution:

🕒 ARCHITECTURE / CONCEPT FLOW

```
5
↓
double → 10
↓
add10 → 20
↓
square → 400
```

### 26.37 pipe()

pipe() সাধারণত left-to-right।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const pipe = (...functions) => value =>
  functions.reduce(
    (result, fn) => fn(result),
    value
  );
```

ব্যবহার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const process = pipe(
  double,
  add10,
  square
);

console.log(process(5));
```

▶ CONSOLE OUTPUT

OUTPUT

400

### 26.38 compose() vs pipe()

🕒 ARCHITECTURE / CONCEPT FLOW

compose:  
right → left

pipe:  
left → right

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
compose(square, add10, double)
```

Conceptually:

🕒 ARCHITECTURE / CONCEPT FLOW

double → add10 → square

অথবা:

JAVASCRIPT SOURCE

JAVASCRIPT

```
pipe(double, add10, square)
```

সরাসরি:

🕒 ARCHITECTURE / CONCEPT FLOW

double → add10 → square

## 26.39 Memoization

**Memoization** হলো কোনো expensive function-এর result cache করে রাখা। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function square(n) {  
  console.log("Calculating ...");  
  return n * n;  
}
```

একই input বারবার দিলে বারবার calculation হচ্ছে। Memoization দিয়ে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function memoize(fn) {  
  
  const cache = new Map();  
  
  return function(value) {  
  
    if (cache.has(value)) {  
      return cache.get(value);  
    }  
  
    const result = fn(value);  
  
    cache.set(value, result);  
  
    return result;  
  };  
}
```

ব্যবহার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const memoizedSquare = memoize(n => {  
  console.log("Calculating ...");  
  return n * n;  
});  
  
console.log(memoizedSquare(5));  
console.log(memoizedSquare(5));
```

► CONSOLE OUTPUT

OUTPUT

```
Calculating ...  
25  
25
```

দ্বিতীয়বার calculation হয়নি।

## 26.40 Memoization কীভাবে কাজ করে?

প্রথমবার:

🕒 ARCHITECTURE / CONCEPT FLOW

```
5  
↓  
Calculation  
↓  
25  
↓  
Cache
```

দ্বিতীয়বার:

🕒 ARCHITECTURE / CONCEPT FLOW

```
5  
↓  
Cache আছে?  
↓  
Yes  
↓  
25
```

#### 26.41 Memoization কোথায় Useful?

যেখানে calculation expensive:

- mathematical calculations
- recursive algorithms
- expensive transformations
- repeated API-derived calculations
- data processing

তবে cache সবসময় memory ব্যবহার করে—তাই অথথা memoization করা উচিত নয়।

#### 26.42 Decorator Pattern

একটি function-এর মূল behavior না বদলে তার চারপাশে additional behavior যোগ করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function greet(name) {  
  return `Hello ${name}`;  
}
```

Decorator:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function logFunction(fn) {  
  
  return function( ... args) {  
  
    console.log("Function called");  
  
    const result = fn( ... args);  
  
    console.log("Function finished");  
  
    return result;  
  };  
}
```

ব্যবহার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const loggedGreet = logFunction(greet);  
  
console.log(loggedGreet("Ripon"));
```

► CONSOLE OUTPUT

OUTPUT

```
Function called  
Function finished  
Hello Ripon
```

#### 26.43 Decorator-এর Real-Life Use

Decorator-style wrapper দিয়ে করা যায়:

##### ► CONSOLE OUTPUT

OUTPUT

```
Logging
Authentication
Timing
Caching
Validation
Permission checking
Error handling
```

যেমন:

##### JAVASCRIPT SOURCE

JAVASCRIPT

```
function withLogging(fn) {
  return (...args) => {
    console.log("Arguments:", args);

    const result = fn(...args);

    console.log("Result:", result);

    return result;
  };
}
```

#### 26.44 Function Timing

##### JAVASCRIPT SOURCE

JAVASCRIPT

```
function withTiming(fn) {
  return (...args) => {
    const start = performance.now();

    const result = fn(...args);

    const end = performance.now();

    console.log(
      `Execution time: ${end - start}ms`
    );

    return result;
  };
}
```

ব্যবহার:

##### JAVASCRIPT SOURCE

JAVASCRIPT

```
const calculate = withTiming(() => {
  let total = 0;

  for (let i = 0; i < 1000000; i++) {
    total += i;
  }

  return total;
});

console.log(calculate());
```

এখানে wrapper function execution time measure করছে।

## 26.45 Debouncing

Debouncing খুব গুরুত্বপূর্ণ web development concept। ধরো search box:

### ▶ CONSOLE OUTPUT

OUTPUT

User types:

R  
Ri  
Rip  
Ripo  
Ripon

প্রতিটি keypress-এ API call করলে:

🕒 ARCHITECTURE / CONCEPT FLOW

5 keystrokes  
↓  
5 API requests

এটা unnecessary হতে পারে। Debounce বলে: > User typing থামানোর পর function execute করে। Basic implementation:

### JAVASCRIPT SOURCE

JAVASCRIPT

```
function debounce(fn, delay) {  
  let timer;  
  
  return function( ... args) {  
    clearTimeout(timer);  
  
    timer = setTimeout(() => {  
      fn( ... args);  
    }, delay);  
  };  
}
```

## 26.46 Debounce Example

### JAVASCRIPT SOURCE

JAVASCRIPT

```
const search = debounce(function(query) {  
  console.log("Searching:", query);  
}, 500);
```

User:

### ▶ CONSOLE OUTPUT

OUTPUT

R  
Ri  
Rip  
Ripo  
Ripon

দ্রুত type করলে previous timers cancel হবে। শেষে:

### ▶ CONSOLE OUTPUT

OUTPUT

Searching: Ripon

একবার execute হতে পারে। React search box-এ এই concept খুব usefull।

## 26.47 Throttling

**Throttling** মানে নির্দিষ্ট সময়ের মধ্যে function সর্বোচ্চ একটি নির্দিষ্ট frequency-তে চালানো। যেমন scroll event:

🕒 ARCHITECTURE / CONCEPT FLOW

```
User scroll করছে খুব দ্রুত
↓
অনেক scroll events
↓
Throttle
↓
নির্দিষ্ট interval-এ function
```

Basic:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function throttle(fn, delay) {
  let waiting = false;
  return function( ... args) {
    if (waiting) return;
    fn( ... args);
    waiting = true;
    setTimeout(() => {
      waiting = false;
    }, delay);
  };
}
```

## 26.48 Debounce vs Throttle

| Debounce                | Throttle                     |
|-------------------------|------------------------------|
| শেষ event-এর পর execute | নির্দিষ্ট interval-এ execute |
| Search input            | Scroll                       |
| Resize শেষে             | Mouse movement               |
| Typing                  | Continuous events            |

সহজভাবে:

► CONSOLE OUTPUT

OUTPUT

```
Debounce:
"থামলে কাজ করো"

Throttle:
"চলতে থাকলেও নির্দিষ্ট সময় পরপর কাজ করো"
```

## 26.49 Closure + Debounce

খেয়াল করো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function debounce(fn, delay) {  
  
  let timer;  
  
  return function( ... args) {  
    clearTimeout(timer);  
  
    timer = setTimeout(() => {  
      fn( ... args);  
    }, delay);  
  };  
}
```

এখানে returned function:

JAVASCRIPT SOURCE

JAVASCRIPT

```
return function( ... )
```

`timer` variable-টিকে মনে রাখছে। এটাই Closure-এর practical power!

## 26.50 Closure + Private Data

আরেকটি strong example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createBankAccount(balance) {  
  
  return {  
    deposit(amount) {  
      balance += amount;  
    },  
  
    withdraw(amount) {  
  
      if (amount ≤ balance) {  
        balance -= amount;  
      }  
    },  
  
    getBalance() {  
      return balance;  
    }  
  };  
}
```

এখানে:

► CONSOLE OUTPUT

OUTPUT

```
balance
```

private! শুধু methods-এর মাধ্যমে access করা যাচ্ছে।

## 26.51 Encapsulation

**Encapsulation** মানে data এবং সেই data-এর operations-কে একসাথে রাখা এবং internal implementation hide করা। Closure দিয়ে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const account = createBankAccount(1000);
```

User জানে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
account.deposit(500);  
account.withdraw(200);  
account.getBalance();
```

কিন্তু internal:

► CONSOLE OUTPUT

OUTPUT

```
balance
```

কীভাবে রাখা হয়েছে সেটা বাইরে থেকে directly manipulate করা যায় না।

## 26.52 Closure Memory Consideration

Closure outer variables ধরে রাখতে পারে। তাই unnecessary বড় data ধরে রাখলে memory usage বাড়তে পারে। Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createSomething() {  
  const hugeData = new Array(1000000).fill("data");  
  
  return function() {  
    console.log(hugeData.length);  
  };  
}
```

Returned function `hugeData` -এর reference ধরে রাখছে। তাই closure ব্যবহার করার সময়: > প্রয়োজনের চেয়ে বেশি data unnecessarily ধরে রাখা উচিত নয়।

## 26.53 Closure-এর Common Mistakes

Closure ব্যবহারের সময় সচরাচর যেসব ভুল এবং ব্যাড-প্র্যাকটিস দেখা যায়:

### ♦ Mistake 1 – গ্লোবাল স্টেট পরিবর্তন (Variable Mutation)

✗ ভুল: গ্লোবাল ভ্যারিয়েবল মিউটেশন

✗ TRAP / ANTIPATTERN

```
let count = 0; // ✗ Global variable leakage

function increment() {
  count++;
}
```

গ্লোবাল স্টেট বাইরের যেকোনো কোড দিয়ে অনিচ্ছাকৃতভাবে বদলে যেতে পারে।

সঠিক সমাধান: ফাংশন স্কোপের ভেতরে প্রাইভেট স্টেট হিসেবে রাখা:

✓ সঠিক: CLOSURE ENCAPSULATED STATE

✓ BEST PRACTICE

```
function createCounter() {
  let count = 0; // ✓ Private lexical state
  return () => ++count;
}
```

### ♦ Mistake 2 – লুপে var ব্যবহারের ফাঁদ

✗ ভুল: VAR লুপ ট্র্যাপ

✗ TRAP / ANTIPATTERN

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // ✗ Output: 3, 3, 3
}
```

► CONSOLE OUTPUT

OUTPUT

```
3
3
3
```

Modern JavaScript সমাধান: let দিয়ে পার-ইটারেশন ব্লক স্কোপিং নিশ্চিত করা:

✓ সঠিক: LET ব্লক স্কোপিং

✓ BEST PRACTICE

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // ✓ Output: 0, 1, 2
}
```

► CONSOLE OUTPUT

OUTPUT

```
0
1
2
```

## 26.54 Advanced Real-Life Example — Permission Factory

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createPermission(role) {
  return function(action) {
    if (role === "admin") {
      return true;
    }

    if (role === "editor" && action === "edit") {
      return true;
    }

    return false;
  };
}
```

ব্যবহার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const adminPermission = createPermission("admin");
const editorPermission = createPermission("editor");

console.log(adminPermission("delete"));
console.log(editorPermission("edit"));
console.log(editorPermission("delete"));
```

► CONSOLE OUTPUT

OUTPUT

```
true
true
false
```

এখানে `role` closure-এর মাধ্যমে মনে রাখা হয়েছে।

## 26.55 Advanced Real-Life Example — ID Generator

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createIdGenerator() {
  let id = 0;

  return function() {
    id++;
    return id;
  };
}

const generateId = createIdGenerator();

console.log(generateId());
console.log(generateId());
console.log(generateId());
```

► CONSOLE OUTPUT

OUTPUT

```
1
2
3
```

এ ধরনের pattern application-level temporary state তৈরি করতে কাজে লাগতে পারে।

## 26.56 Complete Mini Project — Counter

### HTML

HTML STRUCTURE

JAVASCRIPT

```
<button id="increase">+</button>
<button id="decrease">-</button>
<button id="reset">Reset</button>

<h2 id="display">0</h2>
```

### JavaScript

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createCounter() {
  let count = 0;

  return {
    increase() {
      count++;
    },
    decrease() {
      count--;
    },
    reset() {
      count = 0;
    },
    getValue() {
      return count;
    }
  };
}

const counter = createCounter();

const display = document.getElementById("display");

document.getElementById("increase").addEventListener("click", () => {
  counter.increase();
  display.textContent = counter.getValue();
});

document.getElementById("decrease").addEventListener("click", () => {
  counter.decrease();
  display.textContent = counter.getValue();
});

document.getElementById("reset").addEventListener("click", () => {
  counter.reset();
  display.textContent = counter.getValue();
});
```

এখানে:

🕒 ARCHITECTURE / CONCEPT FLOW

```
createCounter()
  ↓
private count
  ↓
methods
  ↓
UI
```

এটি Closure + Encapsulation-এর real-world example।

## 26.57 Closure বনাম Class

Private state দুভাবেই করা যায়।

### ◆ Closure

JAVASCRIPT SOURCE

JAVASCRIPT

```
function createUser(name) {  
  
    return {  
        getName() {  
            return name;  
        }  
    };  
}
```

### ◆ Class

Modern JavaScript class-এ private field:

JAVASCRIPT SOURCE

JAVASCRIPT

```
class User {  
  
    #name;  
  
    constructor(name) {  
        this.#name = name;  
    }  
  
    getName() {  
        return this.#name;  
    }  
}
```

দুটোরই use case আছে।

## 26.58 Closure বনাম Global Variable

### ◆ Global

JAVASCRIPT SOURCE

JAVASCRIPT

```
let count = 0;  
  
function increment() {  
    count++;  
}
```

যে code access করতে পারে, state modify করার সুযোগ পেতে পারে।

### ◆ Closure

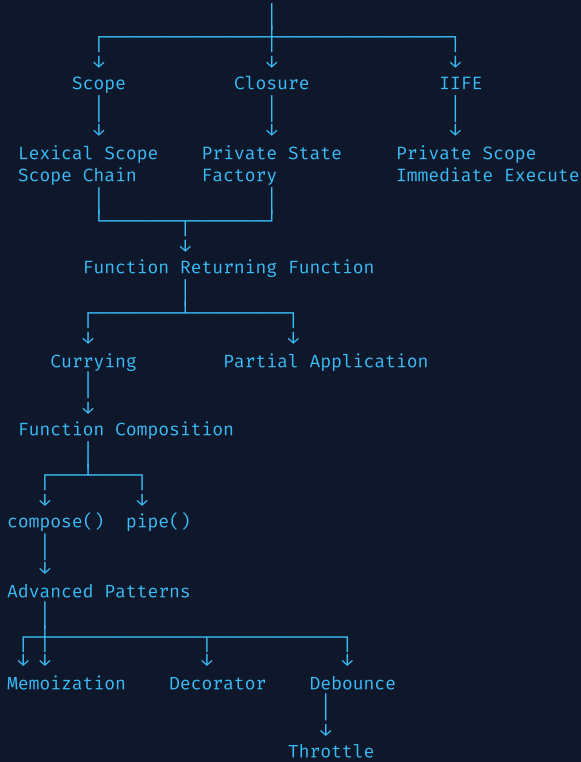
JAVASCRIPT SOURCE

JAVASCRIPT

```
function createCounter() {  
  
    let count = 0;  
  
    return () => ++count;  
}
```

`count` encapsulated থাকে।

## Advanced Functions



## 26.Must 🔥 Must Know — Chapter 26 এর সবচেয়ে গুরুত্বপূর্ণ কনসেপ্টসমূহ

Chapter 26 থেকে ইন্টারভিউ ও প্রোডাকশন কোডিংয়ের জন্য অপরিহার্য মূল বিষয়গুলোর সারসংক্ষেপ:

## ♦ Scope &amp; Lexical Environment

- **Scope:** ভ্যারিয়েবল ও ফাংশনের ভিজিবিলাটি বা এক্সেসিবিলাটি বাউন্ডারি।
- **Lexical Scope:** কোড যেখানে লিখিত হয়েছে, সেই ফিজিক্যাল লোকেশনের ওপর স্কোপ নির্ধারিত হয়।
- **Scope Chain:** কারেন্ট স্কোপ থেকে প্যারেন্ট স্কোপ হয়ে গ্লোবাল পর্যন্ত আইডেন্টিফায়ার খোঁজার চেইন।

## ♦ Closure &amp; Encapsulation

- **Closure:** একটি ফাংশন ও তার লেক্সিক্যাল এনভায়রনমেন্টের রেফারেন্স সংরক্ষণ করে রাখার ক্ষমতা।
- **Private Variables:** বাইরে থেকে অ্যাক্সেস অযোগ্য ডেটা এনক্যাপসুলেশন তৈরি।
- **Function Factory:** প্যারামিটার পাস করে স্পেশলাইজড ডায়নামিক ফাংশন জেনারেশন।
- **Async Closure:** `setTimeout`, ইভেন্ট লিসেনার ও ডিবাউন্সারে স্টেট ধরে রাখা।

## ♦ Immediately Invoked Function Expressions (IIFE)

- **IIFE Syntax:** `(function() { ... })();` বা অ্যারো সিনটাক্স `((() => { ... })());`
- **Private Scope:** গ্লোবাল নেমস্পেসকে পলিউশন থেকে রক্ষা করা এবং ইনিশিয়লাইজেশন কোড একবারে চালানো।

## ♦ Advanced Function Patterns

- **Currying:** `f(a, b, c)` -কে রূপান্তর করে `f(a)(b)(c)` এ একবারে একটি আর্গুমেন্ট গ্রহণকারী ফাংশন চেইন বানানো।
- **Partial Application:** কিছু আর্গুমেন্ট আগে বাইন্ড করে নতুন স্পেশলাইজড ফাংশন তৈরি করা।
- **Composition:** `compose(f, g)(x) = f(g(x))` (Right to Left ডেটা ট্রান্সফরমেশন)।
- **Pipe:** `pipe(f, g)(x) = g(f(x))` (Left to Right স্বাভাবিক পঠনযোগ্য ডেটা পাইপলাইন)।
- **Memoization:** পূর্বের ইনপুট ও ফলাফলের ক্যাশ সংরক্ষণ করে কর্মক্ষমতা অপ্টিমাইজেশন।
- **Debounce & Throttle:** উচ্চ-ফ্রিকোয়েন্সি ইভেন্টকে রेट-লিমিট ও অপ্টিমাইজ করার ক্লোজার প্যাটার্ন।

## 26.Cheat 🍷 Quick Cheat Sheet — Code Pattern Reference

Chapter 26-এর ৯টি প্রধান ফাংশনাল পাটার্নের আল্ট্রা-কম্প্যাক্ট প্রোডাকশন রেফারেন্স:

### 1. CLOSURE (STATE RETENTION)

JAVASCRIPT

```
// Closure
function outer() {
  let value = 10;

  return function() {
    return value;
  };
}
```

### 2. IIFE (IMMEDIATE EXECUTION)

JAVASCRIPT

```
// IIFE
(function() {
  console.log("Run");
})();
```

### 3. CURRYING (UNARY CHAINING)

JAVASCRIPT

```
// Currying
const add = a => b => a + b;

add(10)(20);
```

### 4. FUNCTION FACTORY

JAVASCRIPT

```
// Function Factory
const multiplyBy = x => y => x * y;

const double = multiplyBy(2);
```

### 5. COMPOSE() (RIGHT TO LEFT)

JAVASCRIPT

```
// Composition
const compose = (f, g) => x => f(g(x));
```

### 6. PIPE() (LEFT TO RIGHT)

JAVASCRIPT

```
// Pipe
const pipe = (...fns) => x =>
  fns.reduce((v, fn) => fn(v), x);
```

### 7. MEMOIZATION (CACHE STORE)

JAVASCRIPT

```
// Memoization
const cache = new Map();
```

### 8. DEBOUNCE PATTERN

JAVASCRIPT

```
// Debounce
clearTimeout(timer);
timer = setTimeout(fn, delay);
```

### 9. THROTTLE PATTERN

JAVASCRIPT

```
// Throttle
if (waiting) return;
```

## 26. Lab 📁 Hands-On Practice Lab — 7 Production Mastery Challenges

Closure, Currying, Composition এবং Rate Limiting কনসেপ্টগুলোকে সম্পূর্ণ আয়ত্তে আনতে নিচে ৭টি প্র্যাকটিক্যাল চ্যালেঞ্জের সম্পূর্ণ মডেল সলিউশন দেওয়া হলো:

### Practice 1: Counter Closure — Private State Management

**Problem Requirement:** এমন `createCounter()` ফাংশন তৈরি করো যাতে প্রতিবার কল করলে 1, 2, 3 রিটার্ন করে এবং ভেতরের কাউন্ট বাইরে থেকে মডিফাই করা না যায়।

#### PRACTICE 1 SOLUTION

✅ BEST PRACTICE

```
function createCounter() {
  let count = 0; // Private state
  return function() {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

#### ▶ CONSOLE OUTPUT

OUTPUT

```
1
2
3
```

### Practice 2: Private Balance — Encapsulated Bank Account

**Problem Requirement:** `createAccount(5000)` ফাংশন তৈরি করো যার মধ্যে `deposit(amount)`, `withdraw(amount)` এবং `getBalance()` মেথড থাকবে।

#### PRACTICE 2 SOLUTION

✅ BEST PRACTICE

```
function createAccount(initialBalance) {
  let balance = initialBalance; // Encapsulated private balance

  return {
    deposit(amount) {
      balance += amount;
      return balance;
    },
    withdraw(amount) {
      if (amount ≤ balance) {
        balance -= amount;
      } else {
        console.log("Insufficient funds");
      }
      return balance;
    },
    getBalance() {
      return balance;
    }
  };
}

const account = createAccount(5000);
account.deposit(1000);
account.withdraw(500);
console.log("Final Balance:", account.getBalance());
```

#### ▶ CONSOLE OUTPUT

OUTPUT

```
Final Balance: 5500
```

### Practice 3: Multiplier Factory — Higher-Order Function

**Problem Requirement:** একটি `createMultiplier(multiplier)` ফাংশন ফ্যাক্টরি তৈরি করো যা নতুন মাল্টিপ্লায়ার ফাংশন জেনারেট করে।

#### PRACTICE 3 SOLUTION

✓ BEST PRACTICE

```
const createMultiplier = multiplier ⇒ number ⇒ number * multiplier;

const triple = createMultiplier(3);
const quadruple = createMultiplier(4);

console.log("Triple 10:", triple(10));
console.log("Quadruple 5:", quadruple(5));
```

#### ▶ CONSOLE OUTPUT

OUTPUT

```
Triple 10: 30
Quadruple 5: 20
```

### Practice 4: Currying Function — Unary Parameter Pipeline

**Problem Requirement:** কারিং স্টাইলে `add(10)(20)(30)` কল করে 60 আউটপুট পাওয়ার মতো ফাংশন তৈরি করো।

#### PRACTICE 4 SOLUTION

✓ BEST PRACTICE

```
const add = a ⇒ b ⇒ c ⇒ a + b + c;

const result = add(10)(20)(30);
console.log("Curried Add Result:", result);

// Partial utilization
const add10 = add(10);
const add10And20 = add10(20);
console.log("Partially Applied Result:", add10And20(30));
```

#### ▶ CONSOLE OUTPUT

OUTPUT

```
Curried Add Result: 60
Partially Applied Result: 60
```

### Practice 5: Pipeline Composition — Left-to-Right Data Flow

**Problem Requirement:** `double`, `add10`, `square` তিনটি ফাংশনকে `pipe()` দিয়ে যুক্ত করে 5 থেকে 400 ফলাফল তৈরি করো (5 → 10 → 20 → 400)।

#### PRACTICE 5 SOLUTION

✓ BEST PRACTICE

```
const pipe = (... fns) ⇒ initialVal ⇒
  fns.reduce((val, fn) ⇒ fn(val), initialVal);

const double = x ⇒ x * 2;
const add10 = x ⇒ x + 10;
const square = x ⇒ x * x;

const compute = pipe(double, add10, square);

console.log("Pipeline Output (5 → 10 → 20 → 400):", compute(5));
```

#### ▶ CONSOLE OUTPUT

OUTPUT

```
Pipeline Output (5 → 10 → 20 → 400): 400
```

## Practice 6: Memoization Function — Cache Optimization

**Problem Requirement:** এমন `memoize(fn)` ফাংশন তৈরি করো যাতে একই আর্গুমেন্ট দিলে পূর্বে ক্যাশ করা মান ফেরত আসে এবং পুনরায় ক্যালকুলেশন না হয়।

### PRACTICE 6 SOLUTION

✓ BEST PRACTICE

```
function memoize(fn) {
  const cache = new Map();
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) {
      console.log("[Cache Hit] Returning cached result for:", key);
      return cache.get(key);
    }
    console.log("[Computing] Calculating fresh result for:", key);
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

const slowSquare = n => n * n;
const memoizedSquare = memoize(slowSquare);

console.log(memoizedSquare(12)); // Computing
console.log(memoizedSquare(12)); // Cache Hit
```

### ► CONSOLE OUTPUT

OUTPUT

```
[Computing] Calculating fresh result for: [12]
144
[Cache Hit] Returning cached result for: [12]
144
```

## Practice 7: Debounce Function — Rate Limiting Keystrokes

**Problem Requirement:** একটি `debounce(fn, delay)` ফাংশন তৈরি করো যা ইউজার টাইপিং থামানোর 500ms পর এক্সিকিউট হবে।

### PRACTICE 7 SOLUTION

✓ BEST PRACTICE

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer); // Cancel previous timer
    timer = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

const performSearch = debounce(query => {
  console.log("Searching:", query);
}, 500);

// Simulating rapid user typing
performSearch("R");
performSearch("Ri");
performSearch("Rip");
performSearch("Ripon"); // Only this triggers after 500ms
```

### ► CONSOLE OUTPUT

OUTPUT

```
Searching: Ripon
```

## 26.Map 🧠 Final Mental Map & Core Architecture Flow

JavaScript Advanced Functions-এর মানসিক মডেল এবং কনসেপ্টসমূহের প্রাকৃতিক সংযোগ প্রবাহ:

### 🕒 ADVANCED FUNCTIONS ARCHITECTURE & PARADIGM MAP

```
graph TD
    Scope --> Lexical_Scope[Lexical Scope]
    Lexical_Scope --> Closure
    Closure --> Private_State[Private State]
    Private_State --> Function_Factory[Function Factory]
    Function_Factory --> IIFE
    IIFE --> Function_Returning_Function[Function Returning Function]
    Function_Returning_Function --> Currying
    Currying --> Partial_Application[Partial Application]
    Partial_Application --> Composition
    Composition --> Memoization
    Memoization --> Decorator
    Decorator --> Debounce_Throttle[Debounce / Throttle]
```

### ★ এক নজরে Chapter 26 এর মূল টেক-অ্যাগুয়ে

**Key Takeaway:** JavaScript-এ ফাংশন কেবল কোড এক্সিকিউট করার ব্লক নয়—ফাংশন নিজেই প্রথম-শ্রেণীর ডেটা (First-Class Citizen), যা অন্য ফাংশন তৈরি করতে পারে (Higher-Order), নিজের আশপাশের স্টেট সারাজীবন মনে রাখতে পারে (Closure), ডেটাকে সুরক্ষিত এনক্যাপসুলেশনে আবদ্ধ রাখতে পারে (Private State), এবং একাধিক স্বাধীন ফাংশনকে যুক্ত করে শক্তিশালী ও টেন্ডেবল প্রসেসিং পাইপলাইন তৈরি করতে পারে।